

Week 4 - Reproducible pipelines

Summary

In the first week, you built a data preprocessing pipeline that combines multiple data sources into a clean dataset ready for machine learning. Last week, you returned to this dataset, explored it, trained regression models, and integrated model training into your pipeline.

This final assignment wraps up the project by taking your machine learning workflow and making it ready for production. You will add operational components needed to manage experiments, version data assets, and serve predictions through an API.

Introduction

Up to this point, your focus has been on preparing data, understanding it, and training models that can predict bike rental demand reasonably well. In a real production setting, training a model is only one part of the job. Teams also need to keep track of experiments (i.e., a run of the model training pipeline), manage model versions, reproduce results from specific dataset versions, and expose trained models to others systems via an API.

In this exercise, you will extend your machine learning pipeline with core MLOps functionality. You will track model training runs with MLflow, organize and register trained models, load dataset from versioned storage in LakeFS, and serve predictions through an HTTP API.

The goal is to design and implement a simple, coherent workflow that demonstrates the keys ideas behind production-ready machine learning systems.

Goal

The of this assignment is to extend your existing pipeline with essential MLOps components:

- Track training runs as experiments in MLflow.
- Register trained models in the MLflow model registry.
- Load datasets from versioned storage using LakeFS.
- Version the project assets in LakeFS using a simple branching strategy.
- Load the “production” model from the MLflow registry and expose it as API endpoint.

How to approach this assignment

1. Experiment tracking with MLflow

Start by integrating MLflow into your training workflow from Week 3. Each model training should be logged, including model type, key parameters, evaluation metrics, and model

artifacts. Think carefully about how you organize your experiments.

Once you have a model logged in MLflow, register it in the MLflow model registry. Decide how you will identify the candidate models (e.g., production) and how you distinguish between experimental models and models that are ready to be served.

2. Data and asset versioning with LakeFS

Next, move your dataset handling to versioned storage with LakeFS. Instead of assuming a single fixed dataset, your pipeline should read from the LakeFS repository.

Design a simple branching and versioning strategy for the project. Keep it simple and practical. Your strategy should make it possible to answer questions like:

- Which dataset version was used to train this model?
- How do you protect data being merged into main?

3. Serving predictions through an API

Create a small HTTP API (e.g., using FastAPI) that exposes your model through a prediction endpoint. A user should be able to send input data as JSON and receive predicted bike rental demand in response. While designing your API, keep the business case (as described in week 1) in mind.

The API should not depend on a hardcoded local model file. Instead, it should load the model dynamically from the model registry.

4. Bonus: Adapt the pipeline to automatically train new models when the source data changes.

Monitor the data repository, and run the Dagster pipeline on data changes. The result is a fully automated, hands-off, pipeline.

Acceptance criteria

- The training workflow logs experiments to MLflow, including parameters, metrics, and model artifacts.
- A trained model is registered in the model registry.
- Datasets are loaded from a LakeFS repository.
- Data artifacts (assets) are versioned in LakeFS.
- An HTTP API with a prediction endpoint for the “production” model.
- The codebase remains well-structured and readable.

Tools

MLflow: MLflow is an open source platform for managing the machine learning lifecycle. It can be used to log experiment runs, store model (hyper)parameters, save model artifacts, and it provides a central model registry.

LakeFS: LakeFS is a data versioning system that brings Git-like workflows to datasets stored in object storage. It allows you to create branches, commit changes, and work with reproducible versions of the data. Use the lakefs-spec library for working with LakeFS repositories from Python.

Tips

- Keep the MLOps design simple. The purpose is to demonstrate the concepts. Keep the logging and versioning strategies appropriate for the size of the problem.
- Reproducibility is a core requirement in MLOps. You should always be able to explain which version of the data produced which model.
- Document your design choices. In the review you will be questioned about how you organized experiments, branches, and model versions the way you did.